

Particle Swarm Optimisation for Feature Selection in Classification: A Multi-Objective Approach

Bing Xue, *Member, IEEE*, Mengjie Zhang, *Senior Member, IEEE*, and Will N. Browne, *Member, IMechE*,

Abstract—Classification problems often have a large number of features in the datasets, but not all of them are useful for classification. Irrelevant and redundant features may even reduce the performance. Feature selection aims to choose a small number of relevant features to achieve similar or even better classification performance than using all features. It has two main conflicting objectives of maximising the classification performance and minimising the number of features. However, most existing feature selection algorithms treat the task as a single objective problem. This paper presents the first study on multi-objective particle swarm optimisation (PSO) for feature selection. The task is to generate a Pareto front of non-dominated solutions (feature subsets). We investigate two PSO based multi-objective feature selection algorithms. The first algorithm introduces the idea of non-dominated sorting into PSO to address feature selection problems. The second algorithm applies the ideas of crowding, mutation and dominance to PSO to search for the Pareto front solutions. The two multi-objective algorithms are compared with two conventional feature selection methods, a single objective feature selection method, a two-stage feature selection algorithm and three well-known evolutionary multi-objective algorithms on twelve benchmark datasets. Experimental results show that the two PSO based multi-objective algorithms can automatically evolve a set of non-dominated solutions. The first algorithm outperforms the two conventional methods, the single objective method, and the two-stage algorithm. It achieves comparable results with the existing three well-known multi-objective algorithms in most cases. The second algorithm achieves better results than the first algorithm and all other methods mentioned above.

Index Terms—Feature selection, particle swarm optimisation, multi-objective optimisation

I. INTRODUCTION

Classification is an important task in machine learning and data mining, which aims to classify each instance in the dataset into different groups based on the information described by its features. Without prior knowledge, it is difficult to determine which features are useful. As a result, a large number of features are usually introduced to the dataset, which include relevant, irrelevant and redundant features. However, irrelevant and redundant features are not useful for classification and they may even reduce the classification performance due to the large search space known as “the curse of dimensionality” [1]. Feature selection can address this problem by selecting only relevant features for classification. By eliminating/reducing irrelevant and redundant features, feature selection could reduce the number of features, shorten the training time, simplify the learned classifiers, and/or improve the classification performance [2], [3].

Feature selection is a difficult task because there can be complex interaction among features. An individually relevant

(redundant or irrelevant) feature may become redundant (relevant) when working together with other features. Therefore, an optimal feature subset should be a group of complementary features that span over the diverse properties of the classes to properly discriminate them. The feature selection task is challenging also because of the large search space. The size of the search space increases exponentially with respect to the number of available features in the dataset [4]. Therefore, an exhaustive search is practically impossible in most situations. In order to solve this problem, a variety of search methods have been applied to feature selection, such as greedy search based sequential forward selection (SFS) [5] and sequential backward selection (SBS) [6]. However, these feature selection approaches still suffer from a variety of problems, such as stagnation in local optima and high computational cost.

In order to better address feature selection problems, an efficient global search technique is needed. Evolutionary computation (EC) techniques are well-known for their global search ability. Particle swarm optimisation (PSO) [7], [8] is a relatively recent EC technique based on swarm intelligence. Compared with other EC algorithms such as genetic algorithms (GAs) and genetic programming (GP), PSO is computationally less expensive and can converge more quickly. Therefore, PSO has been used as an effective technique in many fields, including feature selection [3], [9], [10].

Generally, feature selection is a multi-objective problem. It has two main objectives, which are to maximise the classification performance (minimise the classification error rate) and minimise the number of features. These two objectives are usually conflicting and the optimal decision needs to be made in the presence of a trade-off between them. Treating feature selection as a multi-objective problem can obtain a set of non-dominated feature subsets to meet different requirements in real-world applications. Although PSO, multi-objective optimisation, and feature selection have been individually investigated frequently, there are very few studies on multi-objective feature selection. Meanwhile, existing feature selection algorithms suffer from the problems of high computational cost and PSO is argued computationally less expensive than other EC techniques. In addition, the use of PSO for multi-objective feature selection has not been investigated.

This work represents the first time that PSO has been applied to multi-objective feature selection. This will require novel methods to be introduced as there is no longer a single basis global solution, but a set of solutions to meet different requirements.

A. Goals

The overall goal of this paper is to develop a PSO based multi-objective feature selection approach to classification with the expectation of achieving a Pareto front of non-dominated solutions, which include a small number of features and achieve a lower classification error rate than using all available features. In order to achieve this goal, we investigate two Pareto front feature selection algorithms based on multi-objective PSO, which are *NSPSOFS* using the idea of non-dominated sorting and *CMDPSOFS* using the ideas of crowding, mutation and dominance. The two feature selection algorithms will be examined and compared with two traditional methods, a single objective algorithm, a two-stage training algorithm and three well-known evolutionary multi-objective algorithms on twelve benchmark datasets with different numbers of features, classes and instances (Details can be seen in Section IV). Specifically, we will investigate

- whether using a standard single objective PSO with the overall classification error rate as the fitness function can select a good feature subset and achieve similar or even better classification performance than using all features, and can outperform the two traditional methods,
- whether the PSO based two-stage training algorithm can further improve the feature subset evolved by the above PSO based single objective algorithm,
- whether NSPSOFS can evolve a Pareto front of non-dominated solutions, which can outperform the two conventional methods, the single objective algorithm, the two-stage algorithm, and three well-known multi-objective algorithms, and
- whether CMDPSOFS can evolve a better Pareto front than NSPSOFS and outperform all other methods mentioned above.

The scope of this work is continuous (standard) PSO rather than binary PSO [11]. Although both versions of PSO have been successfully applied to single objective feature selection [3], [9], [10], [12], [13], binary PSO has potential limitations, such as the position of a particle in binary PSO is updated solely based on the velocity while the position in standard PSO is updated based on both the velocity and current position [14]. Therefore, as the first work on multi-objective PSO for feature selection, we will start with developing a multi-objective feature selection approach using continuous PSO.

B. Organisation

The remainder of the paper is organised as follows. Section 2 provides background information. Section 3 describes the PSO based multi-objective feature selection algorithms. Section 4 describes experimental design and Section 5 presents experimental results with discussions. Section 6 provides conclusions and future work.

II. BACKGROUND

This section provides background about PSO and multi-objective optimisation, and also reviews typical related work on feature selection.

A. Particle Swarm Optimisation (PSO)

PSO is an evolutionary computation technique proposed by Kennedy and Eberhart in 1995 [7], [8]. PSO is motivated by social behaviours such as birds flocking and fish schooling. The underlying phenomenon of PSO is that knowledge is optimised by social interaction in the population where thinking is not only personal but also social.

PSO is based on the principle that each solution can be represented as a particle in the swarm. Each particle has a position in the search space, which is represented by a vector $x_i = (x_{i1}, x_{i2}, \dots, x_{iD})$, where D is the dimensionality of the search space. Particles move in the search space to search for the optimal solutions. So, each particle has a velocity, which is represented as $v_i = (v_{i1}, v_{i2}, \dots, v_{iD})$. During the movement, each particle updates its position and velocity according to the experience of its own and its neighbours. The best previous position of the particle is recorded as the personal best *pbest* and the best position obtained by the population thus far is called *gbest*. Based on *pbest* and *gbest*, PSO searches for the optimal solutions by updating the velocity and the position of each particle according to the following equations:

$$x_{id}^{t+1} = x_{id}^t + v_{id}^{t+1} \quad (1)$$

$$v_{id}^{t+1} = w * v_{id}^t + c_1 * r_1 * (p_{id} - x_{id}^t) + c_2 * r_2 * (p_{gd} - x_{id}^t) \quad (2)$$

where t represents the t th iteration in the evolutionary process. $d \in D$ represents the d th dimension in the search space. w is inertia weight, which is to control the impact of the previous velocities on the current velocity. c_1 and c_2 are acceleration constants. r_1 and r_2 are random values uniformly distributed in $[0, 1]$. p_{id} and p_{gd} denote the elements of *pbest* and *gbest* in the d th dimension. The velocity is limited by a predefined maximum velocity, v_{max} and $v_{id}^{t+1} \in [-v_{max}, v_{max}]$. The algorithm stops when a predefined criterion is met, which could be a good fitness value or a predefined maximum number of iterations.

B. Multi-Objective Optimisation

Multi-objective problems happen wherever optimal decisions need to be taken in the presence of trade-offs between two or more conflicting objectives. Multi-objective optimisation involves minimising or maximising multiple conflicting objective functions. In mathematical terms, the formulae of a minimisation problem with multiple objective functions can be written as follows:

$$\text{minimise } F(x) = [f_1(x), f_2(x), \dots, f_k(x)] \quad (3)$$

subject to:

$$g_i(x) \leq 0, \quad i = 1, 2, \dots, m \quad (4)$$

$$h_i(x) = 0, \quad i = 1, 2, \dots, l \quad (5)$$

where x is the vector of decision variables, $f_i(x)$ is a function of x , k is the number of objective functions to be minimised, $g_i(x)$ and $h_i(x)$ are the constraint functions of the problem.

In multi-objective optimisation, the quality of a solution is explained in terms of trade-offs between conflicting objectives.

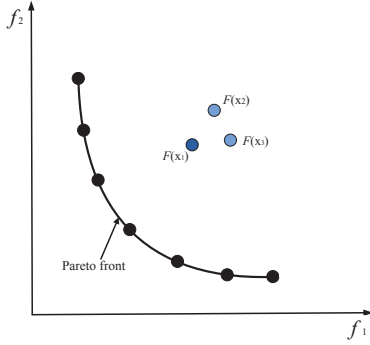


Fig. 1. A minimisation problem with two objective functions.

Let y and z be two solutions of the above k -objective minimisation problem. If the following conditions are met, one can say y dominates z or y is better than z :

$$\forall i : f_i(y) \leq f_i(z) \quad \text{and} \quad \exists j : f_j(y) < f_j(z) \quad (6)$$

where $i, j \in \{1, 2, 3, \dots, k\}$.

Take a two-objective minimisation problem (shown in Figure 1) as an example, x_1 dominates both x_2 and x_3 . For the case that neither x_2 dominates x_3 nor x_3 dominates x_2 , x_2 and x_3 are called non-dominated solutions or trade-off solutions of each other. When a solution is not dominated by any other solutions, it is referred as a Pareto-optimal solution. The set of all Pareto-optimal solutions forms the trade-off surface in the search space, the *Pareto front*. A multi-objective algorithm is designed to search for a set of non-dominated solutions.

Feature selection has two main conflicting objectives, which are minimising both the number of features and the classification error rate. Therefore, feature selection can be expressed as a two-objective minimisation problem.

C. Related Work on Feature Selection

Existing feature selection algorithms can be broadly classified into two categories: wrapper approaches and filter approaches. Wrapper approaches include a learning/classification algorithm in the evaluation procedure while filter approaches do not. Filter approaches are argued to be computationally less expensive and more general while wrapper approaches can usually achieve better results [15]. A number of feature selection algorithms have been proposed in recent years [2]. Typical feature selection algorithms are reviewed in this section.

1) Traditional Feature Selection Approaches

Two commonly used wrapper feature selection methods are SFS [5] and SBS [6]. SFS (SBS) starts with no features (all features), then candidate features are sequentially added to (removed from) the initial feature subset until the further addition (removal) does not increase the classification performance. The limitation of these two methods are that once a feature is selected (eliminated) it cannot be eliminated (selected) later, which is so-called nesting effect [16]. This limitation can be overcome by combining both SFS and SBS into one algorithm. Therefore, the “plus- l -take away- r ” method

is proposed by Stearns [17]. “plus- l -take away- r ” performs l times forward selection followed by r times backward elimination. The challenge is to determine the optimal values of (l, r) . To address this challenge, two floating feature selection algorithms are proposed by Pudil et al. [18], namely sequential forward floating selection (SFFS) and sequential backward floating selection (SBFS). SFFS and SBFS are developed to automatically determine the values for (l, r) . These two floating methods are regarded to be at least as good as the best sequential method, but they also suffer from the problem of stagnation in local optima [16].

The Relief algorithm [19] is a classical filter feature selection algorithm. Relief assigns a weight to each feature to denote the relevance of the feature to the target concept. However, Relief does not deal with redundant features, because it attempts to find all relevant features regardless of the redundancy between them. Decision trees (DT) use only relevant features that are required to completely classify the training set and remove all other features. Cardie [20] proposes a filter feature selection algorithm that uses a DT algorithm to select a subset of features for a nearest neighbour algorithm. The FOCUS algorithm [21], a filter algorithm, exhaustively examines all possible feature subsets, then selects the smallest feature subset. However, the FOCUS algorithm is computationally inefficient because of the exhaustive search.

2) EC Algorithms (Non-PSO) for Feature Selection

Recently, EC techniques have been applied to address feature selection problems, such as GAs, GP and ant colony optimisation (ACO). This subsection briefly review some typical work in the literature.

Based on GAs, Chakraborty [22] proposes a feature selection algorithm using a fuzzy sets based fitness function. However, PSO with the same fitness function in [23] achieve better performance than this GA based algorithm. Hamdani et al. [24] develop a multi-objective feature selection algorithm using non-dominated sorting based multi-objective genetic algorithm II (NSGAI), but the performance of the proposed algorithm has not been compared with any other feature selection algorithm. Based on NSGAI and rough set, Banerjee et al. [25] propose a feature selection method for microarray gene expression data. An initial redundancy reduction of the features is conducted to enable faster convergence and reduce the computational complexity. Although experiments show the effectiveness of the proposed algorithm, only using three datasets is not convincing enough and the results are achieved in a very long evolutionary process (around 15000 generations with 100 individuals).

Zhu et al. [26] propose a hybrid wrapper and filter feature selection algorithm (WFFSA) based on a memetic algorithm, i.e. a combination of GA and local search. In WFFSA, GA adds or deletes a feature based on the ranked individual features. Three different local search strategies, namely improvement first strategy, greedy strategy and sequential strategy are investigated in WFFSA. Experiments show that WFFSA outperforms GA and other methods. This work also shows that a good balance between local search and genetic search can improve the search quality and efficiency of WFFSA.

Muni et al. [27] develop a multi-tree GP algorithm for feature selection (GPmfts) to simultaneously select a feature subset and design a classifier using the selected features. For a c -class problem, each classifier in GPmfts has c trees. Two new crossover operators: homogeneous crossover and heterogeneous crossover are introduced into GPmfts. Comparisons suggest GPmfts achieves better results than SFS, SBS and other methods. However, the number of features selected increases when there are (synthetically added) noisy features. Kourosh and Zhang [28] propose a GP relevance measure (GPRM) to evaluate and rank subsets of features in binary classification tasks, and GPRM is also efficient in terms of feature selection.

Gao et al. [29] propose an ACO based wrapper feature selection algorithm to network intrusion detection. Fisher discrimination rate is adopted as the heuristic information for ACO. Ming [30] proposes a feature selection method based on ACO and rough set. The proposed algorithm starts with the features included in the core of the rough set. Forward selection is adopted into the proposed method to search for the best feature subset. Experimental results show that the proposed algorithm achieves better classification performance with fewer features than a C4.5 based feature selection algorithm. However, experiments do not compare the proposed method with other commonly used feature selection algorithms.

3) PSO Based Feature Selection Approaches

As an EC technique, PSO has recently gained more attention for solving feature selection problems.

Since rough set can handle imprecision, uncertainty and vagueness, Wang et al. [31] propose a filter feature selection algorithm based on an improved binary PSO and rough set. The goodness of a particle is assigned as the dependency degree between class labels and selected features, which is measured by rough set. This work also shows that the computation of the rough set consumes most of the running time, which is a drawback of using rough set in feature selection problems. Fuzzy set can also show the dependency between features and class labels. Chakraborty [23] compares the performance of PSO with that of a GA in a filter feature selection algorithm with a fuzzy set based fitness function. The results show that PSO performs better than GA in terms of the classification performance.

Azevedo et al. [32] propose a wrapper feature selection algorithm using PSO and a support vector machine (SVM) for personal identification in keystroke dynamic system. However, the proposed algorithm obtains a relatively high false acceptance rate, which should be low in most identification systems. Later, Lin et al. [33] propose a wrapper feature selection algorithm (PSO+SVM) using PSO and SVM. The difference from the method in [32] is that PSO+SVM could optimise the parameters in SVM and search for the best feature subset simultaneously. Huang and Dun [13] also propose a similar feature selection method but using two versions of PSO. Binary PSO is used to search for the optimal feature subset and continuous PSO is used to simultaneously optimise the parameters in the kernel function of SVM. Mohammed et

al. [10] propose a hybrid method (PSOAdaBoost) that incorporates PSO with an AdaBoost framework for face detection. PSOAdaBoost aims to search for the best feature subset and determine the decision thresholds of AdaBoost simultaneously, which speeds up the training process and increase the accuracy of weak classifiers in AdaBoost.

The performance of PSO could be improved by properly setting the value of inertia weight to balance its local search and global search. Yang et al. [34] propose two feature selection algorithms, which are based on two inertia weight strategies to properly balance the local search and global search of PSO. The two proposed algorithms outperform other algorithms, such as SFS, “plus- l -take away- r ”, SFFS, a sequential GA and different hybrid GAs.

In standard PSO, g_{best} is updated only when a better solution is founded. Chuang et al. [12] develop a strategy for g_{best} in PSO for feature selection in which g_{best} will be reset to zero if it maintains the same value after several iterations. Chuang et al. [35] apply the so-called catfish effect to PSO for feature selection, which is to introduce new particles into the swarm by initialising the worst particles when g_{best} has not improved for a number of iterations. The introduced catfish particles could help PSO avoid premature convergence and lead to better results than sequential GA, SFS, SFFS and other methods.

Liu et al. [9] introduce a multi-swarm PSO (MSPSO) algorithm to search for the optimal feature subset and optimise the parameters of SVM simultaneously. Experiments show that the proposed feature selection method could achieve higher classification accuracy than grid search, standard PSO and GA. However, the proposed algorithm is computationally more expensive than the other three methods because of the large population size and complicated communication rules between different subswarms. Hamed et al. [36] propose a dynamic quantum-inspired PSO algorithm (DQiPSO) for single objective feature selection and parameter optimisation in neural networks for classification. DQiPSO has two part, where in the first part, the quantum information principle is embedded in PSO as a mechanism for feature probability calculation and on the second part, the standard PSO with real-number encoding is used to optimise the parameters. Experimental results show that the proposed DQiPSO can simultaneously select a good feature subset and optimise the parameters in an artificial neural network. Compared with other two methods (PSO with quantum information principle and standard PSO), the proposed DQiPSO is faster and achieves better classification performance.

Based on PSO, Unler and Murat [3] propose a feature selection algorithm with an adaptive selection strategy, where a feature is chosen not only according to the likelihood calculated by PSO, but also to its contribution to the features already selected. Experiments suggest that the proposed method outperforms the tabu search and scatter search algorithms. Esseghir et al. [37] propose a filter-wrapper feature selection method based on PSO, which aims to integrate the strengths of both filters and wrappers. The proposed filter-wrapper scheme encodes the position of each particle with filter scores of features to reflect feature-class dependency levels. The fitness

of each particle is the classification accuracy achieved by the selected features. Experimental results show that the proposed method could achieve slightly better performance than a binary PSO based filter algorithm. However, the performance of the proposed algorithm has not been compared with that of a wrapper approach, which usually could achieve better results than a filter approach.

A variety of feature selection approaches have been proposed, but most of them aim to minimise the classification error rate only and not much work has been conducted for solving a feature selection task as a multi-objective problem. Although Hamdani et al. [24] develop a NSGAII based multi-objective algorithm, there is no comparisons to test its performance. Many studies have shown that PSO is an efficient search technique for feature selection, but the use of PSO for multi-objective feature selection has not been investigated. Therefore, development of using PSO to propose a multi-objective feature selection algorithm is still an open issue.

III. MULTI-OBJECTIVE APPROACHES

In this section, a commonly used PSO based single objective feature selection algorithm and a PSO based two-stage training feature selection algorithm [38] are firstly described, which are used as the baseline to test the performance of multi-objective algorithms. Then we develop two multi-objective feature selection algorithms using PSO with the goals of selecting a smaller number of features and achieving a lower classification error rate.

A. Two Existing PSO Based Feature Selection Algorithms

1) A Commonly Used PSO Algorithm (ErFS)

PSO has been used as a single objective technique to minimise the classification error rate only [12], [33], [35]. This algorithm is considered in this paper to see whether PSO can do a good job for feature selection. The fitness function is shown in Equation 7, which is to minimise the classification error rate obtained by the selected features during the evolutionary training process.

$$F_1 = \text{ErrorRate} = \frac{FP + FN}{TP + TN + FP + FN} \quad (7)$$

where TP, TN, FP and FN stand for true positives, true negatives, false positives and false negatives, respectively.

The representation of a particle in PSO is a vector of n real numbers, where n is the number of available features in the dataset and also the dimensionality of the search space. A threshold θ is needed to compare with the value (x_{id}) in the position vector. If $x_{id} > \theta$, feature d is selected; otherwise feature d is not selected. This representation is used in all the PSO based algorithms in this paper.

2) PSO with A Two-Stage Fitness Function (2SFS)

The basic fitness function (Equation 7) only considers the classification performance and does not intend to minimise the number of features. Therefore, the feature subset evolved by PSO may still have redundant features and the same classification performance can be achieved by a smaller number of

features. In order to address this problem, a two-stage feature selection approach (2SFS) was proposed [38].

In 2SFS, the whole evolutionary process is divided into two stages. In the first stage, the fitness function is to minimise the classification error rate only. In the second stage, the number of features is taken into account in the fitness function. The second stage starts with the solutions achieved in the first stage, which is expected to ensure that the minimisation of the number of features is based on the achieved feature subsets with high classification performance. The fitness function used in 2SFS is shown in Equation 8.

$$F_2 = \begin{cases} \text{ErrorRate, Stage 1} \\ \alpha * \frac{\#Features}{\#All\ Features} + (1 - \alpha) * \frac{\text{ErrorRate}}{ER}, \text{ Stage 2} \end{cases} \quad (8)$$

where α is a constant value and $\alpha \in [0, 1]$. $\#Features$ stands for the number of features selected. $\#All\ Features$ represents the total number of available features. $ErrorRate$ is the classification error rate obtained by the selected feature subset. ER is the error rate obtained by using all available features for classification on the training set.

In the second stage, both the number of features and the classification performance are considered in the two-stage fitness function (Equation 8), where α shows the relative importance of the number of features and $(1 - \alpha)$ shows the relative importance of the classification performance. As the classification performance is assumed to be more important than the number of features, α is set to be smaller than $(1 - \alpha)$. For fair comparisons, the standard (continuous) PSO is used in 2SFS in this paper while binary PSO is used in [38].

B. PSO Based Multi-Objective Feature Selection Algorithm 1: NSPSOFS

In this section, we investigate a new approach to feature selection using multi-objective PSO with the two main objectives to explore the Pareto front of feature subsets. However, standard PSO was originally proposed for single objective optimisation and could not directly used to address multi-objective problems. In order to investigate a PSO based multi-objective (Pareto front) feature selection algorithm, one of the most important tasks is to determine a good leader ($gbest$) for each particle from a set of potential non-dominated solutions. NSGAII is one of the most popular evolutionary multi-objective techniques [39]. Li [40] introduces the idea of NSGAII into PSO to develop a multi-objective PSO algorithm and achieves promising results on benchmark functions optimisation. However, this algorithm has never been applied to feature selection problems. In this study, we investigate a PSO based multi-objective feature selection algorithm (NSPSOFS) based on the idea of non-dominated sorting in NSGAII to see whether a relatively simple multi-objective PSO can achieve good performance for feature selection problems.

Algorithm 1 shows the pseudo-code of NSPSOFS. The two most important ideas in NSPSOFS are to select a good $gbest$ for each particle and to update the swarm during the evolutionary process. As shown in Algorithm 1, in each iteration, the fitness values (the number of features and the classification error rate) of each particle are calculated. Then

Algorithm 1: Pseudo-Code of NSPSOFS

```

1 begin
2   divide Dataset into a Training set and a Test set;
3   initialise the position and velocity of each particle in the
   swarm (Swarm);
4   while Maximum Iterations is not met do
5     evaluate two objective values of each particle ;
     /* number of features and the
     classification error rate on the
     Training set */
6     identify the particles (nonDomS) that have
     non-dominated solutions in Swarm;
7     calculate crowding distance of each particle in
     nonDomS;
8     Sort particles in nonDomS based on the crowding
     distance;
9     copy all the particles in Swarm to a union (Union);
10    for  $i=1$  to Population Size ( $P$ ) do
11      update the pbest of particle  $i$ ;
12      randomly selecting a gbest for particle  $i$  from the
      highest ranked (least crowded) solutions in
      nonDomS;
13      update the velocity and position of particle  $i$ 
14      add the updated particle  $i$  to Union;
15    end
16    identify different levels of non-dominated fronts
     $F = (F_1, F_2, F_3, \dots)$  in Union;
17    empty the current Swarm for the next iteration;
18     $i = 1$ ;
19    while  $|Swarm| < P$  do
20      if  $(|Swarm| + |F_i| \leq P)$  then
21        add  $F_i$  to Swarm;
22         $i = i + 1$ ;
23      end
24      if  $(|Swarm| + |F_i| > P)$  then
25        calculate crowding distance in  $F_i$ ;
26        sort particles in  $F_i$ ;
27        add the  $(P - |Swarm|)$  least crowded
        particles to Swarm;
28      end
29    end
30  end
31  calculate the classification error rate of the solutions
  (feature subsets) in the  $F_1$  on the test set ; /*  $F_1$  is
  the achieved Pareto front */
32  return the positions of particles in  $F_1$ ;
33  return the training and test classification error rates of the
  solutions in  $F_1$ ;
34 end

```

the algorithm identifies the non-dominated solutions in the swarm according to the fitness values. The crowding distance of each non-dominated solutions is calculated and all the non-dominated solutions are sorted according to the crowding distance. When updating the swarm to a new iteration, a *gbest* for each particle is randomly selected from the highest ranked non-dominated solutions, which are the least crowded solutions. For each particle, *pbest* is replaced with the new position only if the new position dominates the current *pbest*. After determining the *gbest* and *pbest*, the new velocity and the new position of each particle are calculated according to Equations 1 and 2. The old positions (solutions) and the new positions of all particles are firstly combined into one union.

The non-dominated solutions in the union are called the first non-dominated front, which are excluded from the union. Then the non-dominated solutions in the new union are called the second non-dominated front. The following levels of non-dominated fronts are identified by repeating this procedure. For the next iteration, solutions (particles) are selected from the top levels of the non-dominated fronts, starting from the first front (from Line 17 to Line 29). If the number of solutions needed is larger than the number of solutions in the current non-dominated front, all the solutions are added into the next iteration. Otherwise, the solutions in the current non-dominated front are ranked according to the crowding distance and the highest ranked solutions are added into the next iteration.

C. PSO Based Multi-Objective Feature Selection Algorithm 2: CMDPSOFS

NSPSOFS can extend PSO to tackle multi-objective problems. However, NSPSOFS has a potential limitation of losing the diversity of the swarm quickly during the evolutionary process. Specifically, when using the idea of NSGAI to update the population, many of the particles in the new iteration may be identical. Because new particles are selected from the combination of current particles and the updated particles, all non-dominated particles that share the same position will be added into the next iteration. Therefore, the diversity of the swarm might be lost fast during the evolutionary process. In order to better address feature selection problems, we use another multi-objective PSO to develop a multi-objective feature selection algorithm, CMDPSOFS, which is based on the ideas of crowding, mutation and dominance [41] that has never been applied to feature selection problems.

Algorithm 2 shows the pseudo-code of CMDPSOFS. In order to address the main issue of determining a good leader (*gbest*), CMDPSOFS employs a leader set to store the non-dominated solutions as the potential leaders for each particle. A *gbest* is selected from the leader set according to their crowding distances and a binary tournament selection. Specifically, a crowding factor is employed to decide which non-dominated solutions should be added into the leader set and kept during the evolutionary process. The binary tournament selection is used to select two solutions from the leader set and the less crowded solution is chosen as the *gbest*. The maximum size of the leader set is usually set as the number of particles in the swarm. Mutation operators are adopted to keep the diversity of the swarm and improve the search ability of the algorithm. A dominance factor is adopted to determine the size of the archive, which is the number of non-dominated solutions that the algorithm reports. The solutions (feature subsets) in the final archive are used for classification on the test set in each dataset.

Note that CMDPSOFS employs two different mutation operators, uniform mutation in which the variability range allowed for each decision variable is kept constant over generations and non-uniform mutation in which the variability range allowed for each decision variable decreases over time. The two mutation operators are used to maintain the diversity of the swarm and improve the search ability of the

Algorithm 2: Pseudo-Code of CMDPSOFS

```

1 begin
2   divide Dataset into a Training set and a Test set;
   initialise the swarm;
3   initialise the set of leaders LeaderSet and Archive
4   calculate the crowding distance of each member in
   LeaderSet;
5   while Maximum Iterations is not met do
6     for each particle do
7       select a leader (gbest) from LeaderSet for
       each particle by using a binary tournament
       selection based on the crowding distance;
8       update the velocity and position of particle i
9       apply mutation operators;
10      evaluate two objective values for each particle;
        /* number of features and the
        classification error rate on the
        Training set */
11      update the pbest of each particle;
12    end
13    identify the non-dominated solutions (particles) to
    update LeaderSet;
14    send leaders to Archive;
15    calculate the crowding distance of each member in
    LeaderSet;
16  end
17  calculate the classification error rate of the solutions in
  Archive on the test set;
18  return the solutions in Archive and their training and
  test classification error rates;
19 end

```

algorithm. In order to achieve this, CMDPSOFS randomly divides the whole swarm into three different groups in the initialisation procedure. The first group does not have any mutation. The second group employs uniform mutation to keep the global search ability and the third group employs non-uniform mutation to keep the local search ability. Furthermore, the three groups have the same leader set, which allows them to share their success to take advantages of different behaviors to improve the abilities to search for the Pareto non-dominated solutions. Meanwhile, in CMDPSOFS, w is a random value in $[0.1, 0.5]$, c_1 and c_2 are random values in $[1.5, 2.0]$, which are different from most of other PSO based algorithms in which these values are constants. This is used as a convenient way to addressing the problem of tuning these parameters for different test problems.

Both NSPSOFS and CMDPSOFS follow the basic update strategies of standard PSO. As multi-objective algorithms, both NSPSOFS and CMDPSOFS employ a crowding distance to the non-dominated solutions (potential *gbest*) to keep the diversity of the selected *gbest* for particles. The main differences between NSPSOFS and CMDPSOFS are: (1) How to store the non-dominated solutions (potential *gbest*). In NSPSOFS, there is no external set to store non-dominated solutions and all the non-dominated solutions are kept and updated within the swarm. CMDPSOFS includes an external leader set, which is to store the non-dominated solutions. The leader set is updated from iteration to iteration. (2) How to choose a *gbest* for each particle. NSPSOFS ranks all the non-dominated solutions according to their crowding distance, then a *gbest* is randomly selected from the highest ranked (least crowded)

non-dominated solutions. In CMDPSOFS, a binary tournament selection is applied to select two non-dominated solutions from the leader set and the less crowded solution is used as *gbest*. (3) When updating the swarm to a new iteration, NSPSOFS combines the new solutions (after applying the velocity and position update equations) and the old solutions into a union. Different levels of non-dominated solutions are identified from the union to form the swarm in the next iteration. The non-dominated solutions in the last iteration are reported by NSPSOFS. In CMDPSOFS, two different mutation operators are applied together with the update equations and the solutions in the archive are reported as final solutions. (4) Parameters, such as w , c_1 and c_2 , in NSPSOFS are constants and in CMDPSOFS are random values within known ranges.

IV. EXPERIMENTAL DESIGN

A. Benchmark Techniques

In order to examine the performance of the two PSO based multi-objective feature selection algorithms, two conventional wrapper feature selection methods, two single objective algorithms and three well-known evolutionary multi-objective algorithms are used as benchmark techniques in the experiments. The two conventional methods are linear forward selection (LFS) and greedy stepwise backward selection (GSBS). The two single objective algorithms are ErFS and 2SFS described in Section III. The three multi-objective algorithms are NSGAII, strength Pareto evolutionary algorithm 2 (SPEA2) and Pareto archived evolutionary strategy (PAES).

LFS and GSBS were derived from SFS and SBS, respectively. LFS [42] restricts the number of features that are considered in each step of the forward selection, which can reduce the number of evaluations. Therefore, LFS is computationally less expensive than SFS and can obtain good results. More details can be seen in the literature [42].

The greedy stepwise based feature selection algorithm can move either forward or backward in the search space [43]. Given that LFS performs a forward selection, a backward search is chosen in greedy stepwise to conduct a greedy stepwise backward selection (GSBS). GSBS starts with all available features and stops when the deletion of any remaining feature results in a decrease in evaluation, i.e. the accuracy of classification.

NSGAII is one of the most popular evolutionary multi-objective algorithms proposed by Deb et al [39]. The main principle of NSGAII is the use of fast non-dominated sorting technique and the diversity preservation strategy. The fast non-dominated sorting technique is used to rank the parent and offspring populations to different levels of non-dominated solution fronts. A density estimation based on the crowding distance is adopted to keep the diversity of the population. More details can be seen in the literature [39].

SPEA2 is a popular evolutionary multi-objective algorithm proposed by Zitzler et al [44]. The main principle is the fine-grained fitness assignment strategy and the use of an archive truncation method. In SPEA2, the fitness of each individual is the sum of its strength raw fitness and a density estimation. A new population is constructed by the non-dominated solutions

TABLE I
DATASETS

Dataset	# Features	# Classes	# Instances
Wine	13	3	178
Australian	14	2	690
Zoo	17	7	101
Vehicle	18	4	846
German	24	2	1000
World Breast Cancer -Diagnostic (WBCD)	30	2	569
Ionosphere	34	2	351
Lung Cancer	56	3	32
Hillvalley	100	2	606
Musk Version 1 (Musk1)	166	2	476
Madelon	500	2	4400
Isolet5	617	2	1559

in both the original population and the archive. When the number of non-dominated solutions is larger than the population size, the archive truncation method is applied to determine whether a non-dominated solution should be selected or not according to the distance to its k th nearest neighbour. More details can be seen in the literature [44].

PAES is an evolutionary multi-objective algorithm proposed by Knowles and Corne [45]. The authors claimed that PAES may represent the simplest possible non-trivial algorithm capable of generating diverse solutions in the Pareto front. The main idea of PAES is the use of a local search and the use of an archive of previously found non-dominated solutions. PAES was proposed as a baseline approach for Pareto multi-objective algorithms and it has been used to compare with SPEA2 in the literature [44].

B. Datasets and Parameter Settings

Table I shows the twelve datasets used in the experiments, which were chosen from the UCI machine learning repository [46]. They were selected to have various numbers of features (from 13 to 617), classes (from 2 to 7) and instances (from 32 to 4400) and they are used as representative samples of the problems that the PSO based multi-objective feature selection algorithms can address.

In the experiments, all the instances in each dataset are randomly divided into two sets: 70% as the training set and 30% as the test set. During the training process, each particle (individual) represents **one** feature subset. The classification performance of a selected feature subset is evaluated by 10-fold cross-validation on the training set. Note that 10-fold cross-validation is performed as an inner loop in the training process to evaluate the classification performance of a single feature subset on the training set and it does not generate 10 feature subsets. After the training process, the selected features are evaluated on the test set to obtain the testing classification error rate. A detailed discussion of why and how 10-fold cross-validation is applied in this way is given by [15].

All the algorithms are wrapper approaches, i.e. needing a learning algorithm in the evolutionary training process to evaluate the classification performance of the selected feature subset. Any learning algorithm can be used here, such as NB, DT and SVM. One of the simplest and commonly used learning algorithms [34], [35], K-nearest neighbour (KNN), was chosen in the experiments. We use $K=5$ in KNN (5NN)

to simplify the evaluation process and 5NN implemented in Java machine learning library (Java-ML) [47] is used here.

Waikato Environment for Knowledge Analysis (Weka) [48] is used to run the experiments of using LFS and GSBS for feature selection. All the settings in LFS and GSBS are kept to the defaults except that backward search is chosen in the greedy stepwise approach to perform GSBS for feature selection. During the feature selection process, 5NN with 10-fold cross-validation in Weka is employed to evaluate the classification accuracy on the training set. In order to make fair comparisons, all the feature subsets selected by LFS, GSBS and other feature selection methods are tested by 5NN in Java-ML on the test sets.

In all the PSO based algorithms, the fully connected topology is used, the maximum velocity $v_{max} = 0.6$, the population size $P = 30$ and the maximum iteration $T = 100$. In ErFS, 2SFS and NSPSOFS, the inertia weight $w = 0.7298$, the acceleration constants $c_1 = c_2 = 1.49618$. These values are chosen based on the common settings in the literature [8], [49]. In the CMDPSO, w is a random value in $[0.1, 0.5]$, c_1 and c_2 are random values in $[1.5, 2.0]$, and the mutation rate is $1/n$, where n is the number of available features (dimensionality). These values are based on the settings of an equivalent algorithm in the literature [41]. According to our previous experiments, the threshold θ in ErFS, 2SFS, NSPSOFS and CMDPSOFS is set as 0.6. In 2SFS, the first 50 iterations are set as the first stage and the last 50 iterations are the second stage. We assume that the number of features is less important than the classification performance. Therefore, $\alpha = 0.2$ in Equation 8 in the second stage of 2SFS. In NSGAI, SPEA2 and PAES, the representation of each individual is the same as the commonly used GA for feature selection algorithms [22], [24], where each individual is encoded by a n -bit binary string, and n is the number of available features. The bit with value '1' indicates the feature is selected in the subset, and '0' otherwise. A bit-flip mutation operator is applied in three methods and a single point crossover operator is used in NSGAI and SPEA2. The mutation rate is $1/n$, where n is the number of available features (dimensionality) and the crossover probability is 0.9. For each dataset, all the algorithms have been conducted for 40 independent runs, except for LFS and GSBS, because both of them are deterministic methods.

V. RESULTS AND DISCUSSIONS

For each dataset, LFS and GSBS obtain a unique solution (feature subset). ErFS and 2SFS obtain a single solution in each of the 40 independent runs. Multi-objective algorithms, NSPSOFS, CMDPSOFS, NSGAI, SPEA2, and PAES obtain a set of non-dominated solutions in each run. In order to compare these three kinds of results, the single solution in LFS and GSBS, 40 solutions (from 40 runs) resulted from ErFS and 2SFS are presented in this section. 40 sets of feature subsets achieved by one multi-objective algorithm are firstly combined into one union set. In the union set, for the feature subsets including the same number of features (e.g. m), their classification error rates are averaged. The average classification error rate is assigned as the average classification performance of the subsets with m features. Therefore, a

TABLE II
EXPERIMENTAL RESULTS OF LFS, GSBS, ErFS AND 2SFS.

	Wine					Australian				
Method	All	LFS	GSBS	ErFS	2SFS	All	LFS	GSBS	ErFS	2SFS
AveSize	13	7	8	8	8	14	4	12	3.88	3.42
AveEr	23.46	25.93	14.81	4.04	4.04	29.95	29.95	30.43	14.52	15.76
BestEr				0	0				12.56	12.56
StdEr				1.83E-2	1.83E-2				3.6E-2	4.56E-2
	Zoo					Vehicle				
Method	All	LFS	GSBS	ErFS	2SFS	All	LFS	GSBS	ErFS	2SFS
AveSize	17	8	7	9.18	9.18	18	9	16	9.52	8.65
AveEr	19.05	20.95	20	4.5	4.5	16.14	16.93	24.21	15	15.05
BestEr				2.86	2.86				12.99	12.99
StdEr				90.1E-4	90.1E-4				79E-4	77.8E-4
	German					WBCD				
Method	All	LFS	GSBS	ErFS	2SFS	All	LFS	GSBS	ErFS	2SFS
AveSize	24	3	18	13.48	11.92	30	10	25	13.42	5
AveEr	32	31.33	35.67	30.59	30.85	7.02	11.11	16.37	6.61	6.46
BestEr				28	28				5.26	5.26
StdEr				1.33E-2	1.18E-2				55.8E-4	75.3E-4
	Ionosphere					Lung				
Method	All	LFS	GSBS	ErFS	2SFS	All	LFS	GSBS	ErFS	2SFS
AveSize	34	4	30	12.58	12.05	56	6	33	27.35	27.38
AveEr	16.19	13.33	21.9	11.6	11.86	30	10	10	28	27.75
BestEr				6.67	8.57				20	10
StdEr				2.14E-2	1.89E-2				6E-2	6.89E-2
	Hillvalley					Musk1				
Method	All	LFS	GSBS	ErFS	2SFS	All	LFS	GSBS	ErFS	2SFS
AveSize	100	8	90	47.32	47.05	166	10	122	86.48	85.58
AveEr	43.41	35.38	50.55	42.46	42.43	16.08	14.69	23.78	15.42	15.42
BestEr				38.19	38.19				11.19	11.19
StdEr				1.52E-2	1.55E-2				2.04E-2	1.99E-2
	Madelon					Isolet5				
Method	All	LFS	GSBS	ErFS	2SFS	All	LFS	GSBS	ErFS	2SFS
AveSize	500	7	489	258.1	256.48	617	24	560	318.7	315.62
AveEr	29.1	35.38	48.72	23.45	23.48	1.55	1.66	2.84	1.43	1.43
BestEr				20.51	20.64				1.23	1.25
StdEr				1.22E-2	1.26E-2				9.98E-4	9.65E-4

set of average solutions is obtained by using the average classification error rates and the corresponding numbers (e.g. m). The set of average solutions is called the *average* Pareto front and presented here. Besides the average Pareto front, the non-dominated solutions in the union set are also presented to compare with the solutions achieved by single objective algorithms. Since PAES achieved similar results to SPEA2, only the results of NSGAII and SPEA2 are shown in this section.

A. Results of Benchmark Techniques: LFS, GSBS, ErFS and 2SFS

Table II shows the experimental results of LFS, GSBS, ErFS and 2SFS. In the tables, “All” shows the results of using all available features for classification. “AveSize” represents the average size of the feature subsets evolved by PSO in the 40 runs. “AveEr”, “BestEr” and “StdEr” show the average, the smallest and the standard deviation of the test classification error rates in the 40 runs, respectively.

a) *Results of LFS and GSBS*: Based on Table II, in most cases, LFS selected a small number of features and achieved a similar or even lower classification error rate than using all available features. GSBS could reduce the number of features, but only reduced the classification error rate on a few datasets. In most cases, LFS outperformed GSBS in terms of both the number of features and the classification performance. The results suggest that LFS as a forward selection algorithm is more likely to obtain some optimality of the small feature subsets than GSBS (backward selection) because of different starting points. Feature subset selected by GSBS may still have redundancy.

b) *Results of ErFS*: According to Table II, in almost all datasets, PSO with the basic fitness function (ErFS) evolved a feature subset which only contained around half (or less than half in six datasets) of the available features and achieved a lower classification error rate than using all features. All the standard deviation values (StdEr) are smaller than 0.05 except in the Lung dataset, which only has a small number of instances and the classification performance changes more than in a dataset with more instances. All the standard deviation values in Table II are very small, which indicates that all algorithms are considerably stable and statistical significant testing is not necessary here. As the objective functions of LFS, GSBS and ErFS are to minimise the classification error rate only, the results also suggest that PSO as an evolutionary search technique can obtain a better feature subset than LFS (in almost all cases) and GSBS (in all cases).

c) *Results of 2SFS*: According to Table II, 2SFS evolved a feature subset with around half (or less than half) of the available features and achieved a lower classification error rate than using all features in all datasets. Comparing with LFS, 2SFS achieved a lower classification error rate than LFS in most cases and a smaller number of features in some cases. 2SFS outperformed GSBS in terms of both the number of features and the classification performance in almost all datasets. 2SFS evolved a smaller number of features and achieved similar or even better classification performance than ErFS in almost all datasets.

The results in Table II suggest PSO with different fitness functions in ErFS and 2SFS obtain different feature subsets with different numbers of features and classification error rates. 2SFS further improves the feature subset obtained by ErFS in almost all cases. In the first stage, the fitness function (Equation 8) could guide PSO to search for the feature subset with the minimum classification error rate, then in the second stage, it guide PSO to search for the smallest feature subset with the already achieved high classification performance. Therefore, 2SFS can successfully remove redundant features and achieve high classification performance in most cases. However, in the German and Ionosphere datasets, 2SFS could further reduce the number of features, but the classification performances are slightly worse than ErFS. Therefore, Pareto front multi-objective feature selection algorithms are needed to better address the problems.

B. Results of NSPSOFS and CMDPSOFS

As 2SFS usually achieved better performance than the other methods, only the results of 2SFS are included in this section as a baseline to test the performance of NSPSOFS and CMDPSOFS. Figure 2 shows the experimental results of NSPSOFS, CMDPSOFS and 2SFS. In Figure 2, each chart corresponds to one of the dataset used in the experiments. On the top of each chart, the numbers in the brackets show the number of available features and the classification error rate using all features. In each chart, the horizontal axis shows the number of features selected and the vertical axis shows the classification error rate.

In Figure 2, “-A” stands for the average Pareto front resulted from NSPSOFS or CMDPSOFS in the 40 independent runs.

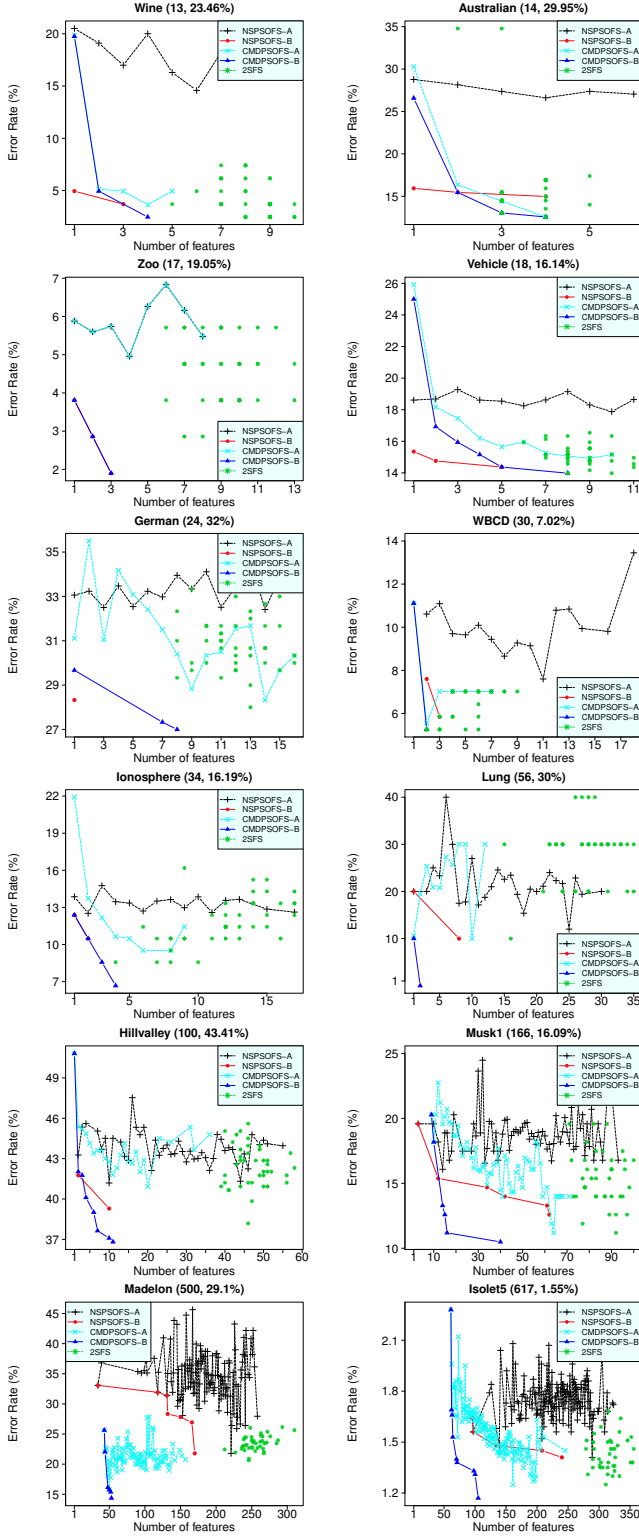


Fig. 2. Experimental Results of NSPSOFS, CMDPSOFS and 2SFS.

“-B” represents the non-dominated solutions resulted from NSPSOFS or CMDPSOFS in the 40 independent runs. “2SFS” shows the 40 solutions of 2SFS. In some datasets, 2SFS may evolve the same feature subset in different runs and they are shown in the same point in the chart. Therefore, although 40 results are presented, there may be less than 40 distinct points shown in a chart.

1) Results of NSPSOFS

As can be seen in Figure 2, in most cases, the average Pareto front of NSPSOFS (NSPSOFS-A) includes two or more solutions, which selected a smaller number of features and achieved a lower classification error rate than using all features. For the same number of features, there are a variety of combinations of features with different classification performances. The feature subsets obtained in different runs may include the same number of features but different classification error rates. Therefore, although the solutions obtained in each run are non-dominated, some solutions in the average Pareto front may dominate others.

According to Figure 2, the non-dominated solutions (NSPSOFS-B) include one or more feature subsets, which achieved better classification performance than using all features in all datasets. In most datasets, NSPSOFS evolved a feature subset only selected 1 or 2 features, but achieved a lower classification error rate than using all features. For example, NSPSOFS selected only around 1.8% of the available features (1 from 56) in the Lung dataset and selected 2% of the available features (2 from 100) in the Hillvalley dataset, but achieved higher classification performance than using all features. In almost all cases, the number of features was reduced to 10% or less, except for around 17.64% in Zoo, 26.4% in Madelon and 21.72% in Isolet5.

Comparing NSPSOFS with 2SFS, it can be seen that classification error rates of 2SFS and the average Pareto front (NSPSOFS-A) are similar in many datasets, but the number of features in NSPSOFS-A is usually smaller than in 2SFS. In almost all cases, NSPSOFS-B included fewer features and achieved a lower classification error rate than 2SFS.

The results suggest that although NSPSOFS shares the same parameter settings as 2SFS, NSPSOFS as a multi-objective technique can effectively explore the Pareto front and obtain feature subsets, which include a smaller number of features and achieve better classification performance than 2SFS.

2) Results of CMDPSOFS

According to Figure 2, in all cases, CMDPSOFS-A includes two or more feature subsets, which successfully selected a smaller number of relevant features and achieved a lower classification error rate than using all features.

As can be seen in Figure 2, CMDPSOFS-B include one or more solutions, which achieved a lower classification error rate than using all features in all datasets. In most datasets, CMDPSOFS evolved a feature subset, which only selected 1 or 2 features, but achieved a better classification performance than using all features. For example, CMDPSOFS selected only around 1.8% of the available features (1 from 56) in the Lung dataset and selected only 2% of the available features (2 from 100) in the Hillvalley dataset, but achieved a lower classification error rate than using all features. In almost all datasets, the number of features selected is less than 10% of the total number of features, except for around 16.67% in the Vehicle dataset and 10.37% in the Isolet5 dataset.

Comparing CMDPSOFS with 2SFS, in all datasets, the classification performance of CMDPSOFS-A is similar to that of

2SFS, but the number of features in CMDPSOFS-A is smaller than in 2SFS. Moreover, CMDPSOFS-B outperformed 2SFS in terms of both the number of features and the classification performance in all datasets.

The results suggest that CMDPSOFS as a multi-objective technique guided by the two objectives can effectively explore the Pareto front and achieve better feature subsets than 2SFS. Note that proper settings of the α value and the division of the two stages in 2SFS might increase its performance on either the number of features or the classification performance, but this requires prior knowledge and is problem specific. This is a disadvantage of 2SFS compared with Pareto front multi-objective algorithms.

C. Comparisons Between NSPSOFS, CMDPSOFS, NSGAII and SPEA2

In order to test the performance of NSPSOFS and CMDPSOFS, they are compared with three popular evolutionary multi-objective algorithms, NSGAII, SPEA2 and PAES. PAES achieved similar results to SPEA2 in terms of the number of features and the classification performance. Therefore, the results of PAES are not presented in this section. Comparisons between NSPSOFS, NSGAII and SPEA2 are shown in Figures 3. Comparisons between CMDPSOFS, NSGAII and SPEA2 are shown in Figure 4.

1) Comparisons Between NSPSOFS, NSGAII and SPEA2

According to Figures 3, in most cases, the average Pareto fronts, NSPSOFS-A, NSGAII-A and SPEA2-A include two or more feature subsets, which selected a smaller number of features but achieved better classification performance than using all features. There are also some dominated solutions in three average Pareto fronts and the reason is the same as discussed in Section V-B.

Comparing NSPSOFS-A with NSGAII-A and SPEA2-A, in most cases, the classification performance of three methods are similar. Although in a few cases the classification error rates of NSPSOFS-A is slightly higher than that of NSGAII-A and SPEA2-A, the number of features is usually smaller in NSPSOFS-A than in NSGAII-A and SPEA2-A.

In terms of the non-dominated solutions (NSPSOFS-B, NSGAII-B and SPEA2-B), results in different datasets show different patterns. Specifically, in three datasets (Zoo, Vehicle and German), NSPSOFS-B dominated NSGAII-B and SPEA2-B while in two datasets (WBCD and Lung), NSPSOFS-B was dominated by NSGAII-B and SPEA2-B. In the other datasets, there are always solutions in NSPSOFS-B, which dominate solutions in NSGAII-B and SPEA2-B although there are also solutions in NSPSOFS-B dominated by solutions in NSGAII-B and SPEA2-B. In most cases, NSGAII-B outperformed SPEA2-B in terms of the classification performance and the number of features.

The results in Figure 3 suggest that NSPSOFS, NSGAII and SPEA2 are generally competitive to each other. However, as discussed in Section III-C, NSPSOFS has a potential limitation of quickly losing the diversity of the swarm because of the updating mechanism. The performance of a PSO based multi-objective algorithm could be improved if this limitation can be addressed.

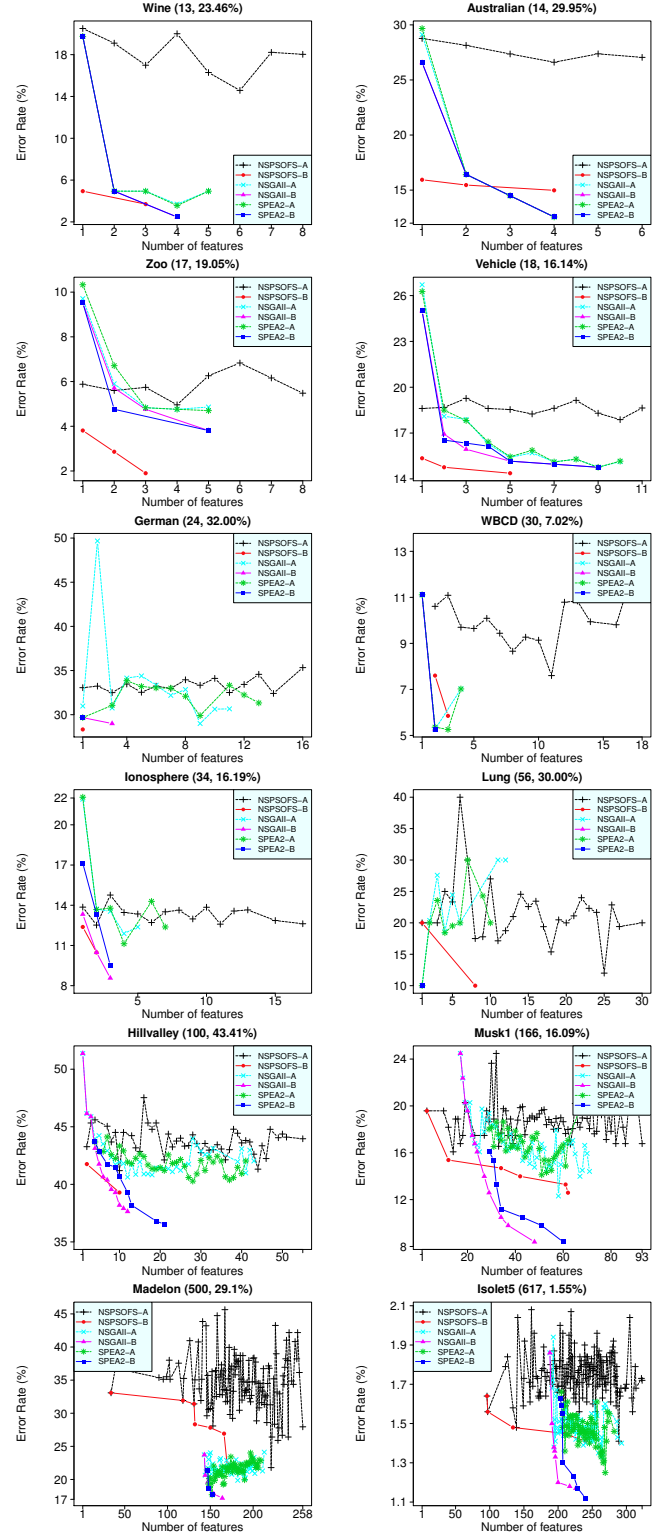


Fig. 3. Comparisons between NSPSOFS, NSGAII and SPEA2.

2) Comparisons Between CMDPSOFS, NSGAII and SPEA2

As can be seen in Figure 4, the average Pareto fronts, CMDPSOFS-A, NSGAII-A and SPEA2-A achieved similar classification performance in all datasets. However, the number of features in CMDPSOFS-A is usually smaller than that of NSGAII-A and SPEA2-A, especially in the Madelon and Isolet5 datasets.

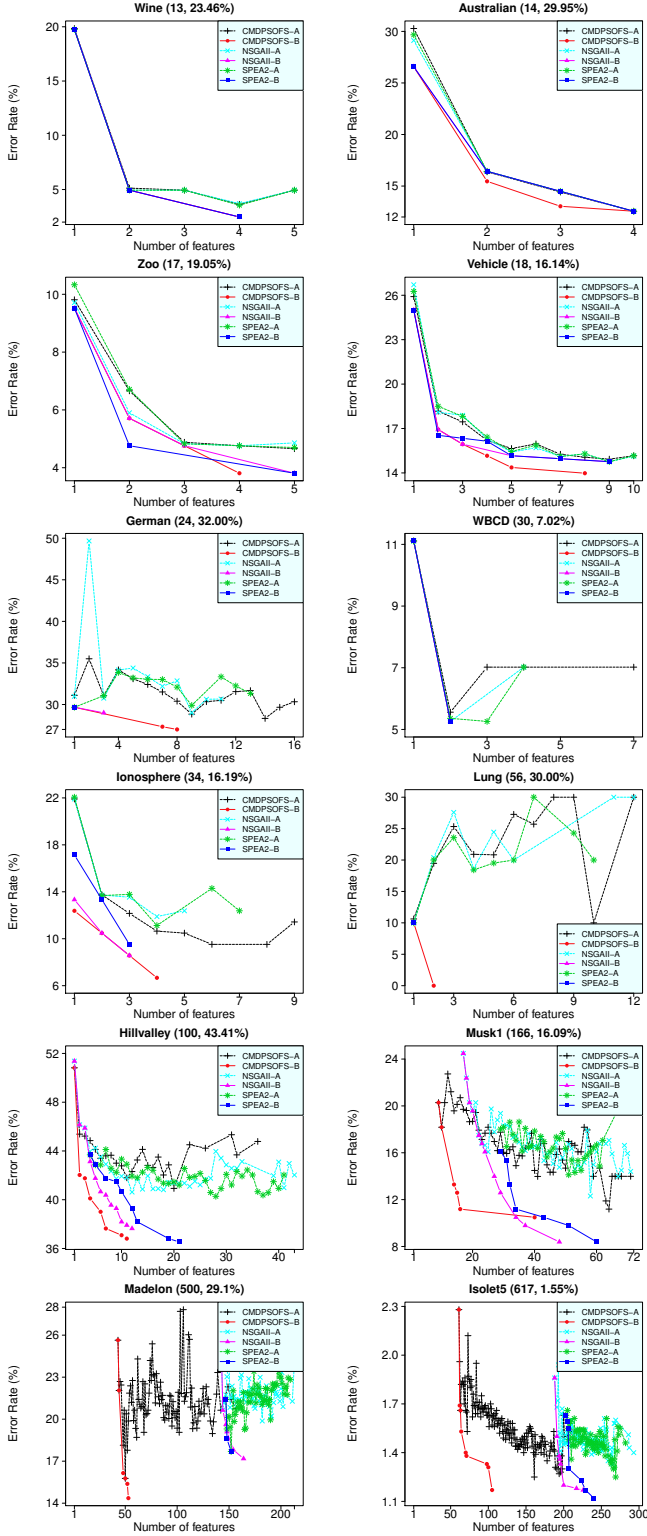


Fig. 4. Comparisons between CMDPSOFS, NSGAII and SPEA2.

Comparing the non-dominated solutions CMDPSOFS-B with NSGAII-B and SPEA2-B, it can be seen that in almost all datasets, CMDPSOFS-B achieved better results than NSGAII-B and SPEA2-B in terms of both the number of features and the classification performance. In a dataset with a large number of features, the better performance of CMDPSOFS is more obvious, especially for the number of features. For example, in

TABLE III
T-TEST ON HYPER VOLUME RATIOS ON TESTING ACCURACY

Dataset	Wine		Australian		Zoo		Vehicle		German		WBCD	
	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD
NSPSOFS		=		-		=		=		=		?
CMDPSOFS	=		+		=		=		=		?	
NSGAII	=	=	+	=	=	=	=	=	=	=	?	?
SPEA2	=	=	+	=	=	=	=	=	=	=	?	?
PAES	=	=	=	=	=	=	=	=	=	=	?	?

Dataset	Lung		Ionosphere		Hillvalley		Musk1		Madelon		Isolet5	
	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD
NSPSOFS		?		+		+		+		=		+
CMDPSOFS	?		-		-		-		=		-	
NSGAII	?	?	-	=	-	+	-	+	=	=	+	+
SPEA2	?	?	=	+	-	+	-	+	=	=	+	+
PAES	?	?	-	=	-	+	-	+	=	=	-	+

the Madelon dataset, the numbers of features in NSGAII-B and SPEA2-B are around 150 while this number in CMDPSOFS-B is only around 50, which means CMDPSOFS further reduced about 67% of the number of features selected.

The results show that CMDPSOFS can address the limitation in NSPSOFS and achieve better performance than NSPSOFS, NSGAII and SPEA2 in terms of both the number of features and the classification performance.

D. Results of Hyper Volume Indicator

In order to further compare the results of multi-objective algorithms, NSPSOFS, CMDPSOFS, NSGAII, SPEA2 and PAES, the hyper volume indicator [50] is used in the experiments. In each run, each method obtained two Pareto fronts, which are a *training Pareto front* according to the training classification performance and the number of features, and a *testing Pareto front* according to the testing classification performance and the number of features. Therefore, for each method, we calculated two sets of hyper volume values based on the Pareto-fronts on the training process and the testing process, respectively. Therefore, for each method, 40 hyper volume values on the training process and 40 hyper volume values on the testing process were calculated. As the calculation of hyper volume needs the true Pareto front, which is not available in the tested datasets, we firstly combine the training (or testing) Pareto front of these five methods into a union, then identify the Pareto front in the union as the “true Pareto front” to calculate the hyper volume values. The hyper volume values are normalised to hyper volume ratios, which is the division of the hyper volume value of a Pareto front and the hyper volume of the “true Pareto front”. In order to compare NSPSOFS and CMDPSOFS with the other three algorithms, NSGAII, SPEA2 and PAES, the Student’s T-test was performed on their hyper volume ratios, where the significance level was set as 0.05 (or confidence interval is 95%).

1) *Results of Hyper Volume on Testing Process*: Table III shows the results of the T-test between NSPSOFS, CMDPSOFS, NSGAII, SPEA2 and PAES on the hyper volume ratios in the *testing* process, where “NS” and “CMD” represent NSPSOFS and CMDPSOFS. In Table III, “+” (“-”) indicates that NSPSOFS or CMDPSOFS is significantly better (worse) than another corresponding algorithm. “=” means they are similar. In the WBCD and Lung datasets, the “?” means the hyper volume ratio could not be obtained because the extracted “true Pareto front” only contains two points and its hyper volume value is zero.

TABLE IV
T-TEST ON HYPER VOLUME RATIOS ON TRAINING ACCURACY

Dataset	Wine		Australian		Zoo		Vehicle		German		WBCD	
	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD
NSPSOFS	-	+	-	+	-	+	-	+	-	+	-	+
CMDPSOFS	-	-	-	-	-	-	-	-	-	-	-	-
NSGAI	-	=	-	=	-	-	-	-	-	=	-	=
SPEA2	-	=	-	=	-	=	-	=	-	=	-	=
PAES	-	=	-	=	-	-	-	-	-	-	-	=

Dataset	Lung		Ionosphere		Hillvalley		Musk1		Madelon		Isolet5	
	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD	NS	CMD
NSPSOFS	-	+	-	+	-	+	-	+	-	+	-	+
CMDPSOFS	-	-	-	-	-	-	-	-	-	-	-	-
NSGAI	-	-	-	-	-	+	-	+	=	+	+	+
SPEA2	-	-	-	-	=	+	-	+	=	+	+	+
PAES	-	-	-	-	-	-	-	-	-	-	-	-

Table III shows that compared with CMDPSOFS, NSGAI, SPEA2 and PAES, NSPSOFS achieved similar results in most cases, although NSPSOFS achieved better results on the Australian dataset and worse results on the Hillvalley and Musk1 datasets. Table III also shows that CMDPSOFS achieved similar results with other methods in most cases. In the datasets with a large number of features, such as Hillvalley, Musk1 and Isolet5, CMDPSOFS achieved significantly better results than NSPSOFS, NSGAI, SPEA2, and PAES.

2) *Results of Hyper Volume on Training Process:* Table IV shows the results of the T-test between NSPSOFS, CMDPSOFS, NSGAI, SPEA2 and PAES on the hyper volume value ratios in the *training* process. It can be seen that NSPSOFS achieved slightly worse results than other methods in most cases, but NSPSOFS achieved better results than NSGAI and SPEA2 in the Isolet5 dataset, where the number of features is large. Table IV also shows that in the datasets with a relatively small number of features, CMDPSOFS usually achieved similar results to NSGAI, SPEA2 and PAES. In the datasets with large numbers of features, such as Hillvalley, Musk1, Madelon and Isolet5, CMDPSOFS achieved significantly better results than NSPSOFS, NSGAI and SPEA2. Although CMDPSOFS achieved slightly worse results than PAES on the training set, CMDPSOFS achieved similar or better results than PAES on the test set (shown in Table III), which is considered due to the overfitting problem in PAES.

E. Comparisons on Computational Time

Table V shows the average computational time (in minutes) used by NSPSOFS, CMDPSOFS, NSGAI, SPEA2 and PAES in one run.

From Table V, it can be seen that for datasets that have a small number of features and instances, SPEA2 and PAES generally use less time than the other three methods. However, all algorithms can perform one run in relatively short time, a few minutes or even less than one minute, such as the Wine, Zoo and Lung datasets. For datasets with a large number of features and instances, CMDPSOFS and PAES used shorter time than the other three methods, especially for the Madelon and Isolet5 datasets, where CMDPSOFS used much less time than NSPSOFS, NSGAI and SPEA2. In such large datasets, computational time is more important than in small datasets. CMDPSOFS can finish the evolutionary training process in much shorter time and achieve better results, which suggests that this method is a better choice than the other four methods

TABLE V
COMPARISONS ON COMPUTATIONAL TIME (IN MINUTES)

	Wine	Australian	Zoo	Vehicle	German	WBCD
NSPSOFS	0.29	4.83	0.11	7.77	12.61	4.34
CMDPSOFS	0.25	4	0.09	6.59	9.3	2.71
NSGAI	0.24	3.99	0.09	6.59	9.32	2.67
SPEA2	0.2	3.24	0.07	5.53	7.64	2.13
PAES	0.21	3.95	0.07	6.35	8.85	2.01

	Lung	Ionosphere	Hillvalley	Musk1	Madelon	Isolet5
NSPSOFS	0.02	1.79	46.04	10.66	868.75	374.63
CMDPSOFS	0.01	1.54	23.49	6.02	394.45	200.71
NSGAI	0.01	1.06	28.88	7.46	721.71	338.23
SPEA2	0.01	0.87	30.23	6.69	694.91	331.91
PAES	0.01	1.04	24.88	5.73	560.83	276.97

in real-world applications, where a large number of features and instances are involved.

All the methods have the same number of evaluations as they have the same number of individuals and iterations during the evolutionary process. NSGAI and NSPSOFS generally consumed more time, which is probably caused by the different levels of non-dominated ranking mechanism and the calculation of crowding distances. CMDPSOFS also involves ranking, but it only happens in the small leader set. Therefore, ranking in CMDPSOFS does not cost as much time as NSGAI and NSPSOFS. More importantly, during the evolutionary training process, CMDPSOFS selected smaller numbers of features than the other four algorithms, which cost much less time for the 10-fold cross-validation to calculate the training classification performance in each evaluation, especially for the datasets with a large number of features.

F. Further Discussions

Experimental results show that the NSPSOFS and CMDPSOFS can be successfully used for feature selection, but NSPSOFS could not achieve as good results as CMDPSOFS. The main reasons are that feature selection problems are difficult problems with many local optima. CMDPSOFS employs different mechanisms to maintain the diversity of both the leader set and the swarm. Specifically, it selects and filters out crowded leaders and uses different mutation operators to maintain the diversity of the swarm to avoid being stagnation in local optima.

As discussed in Section III, CMDPSOFS has an external leader set to store the non-dominated solutions, which are used as potential leaders for each particle. Different from other multi-objective evolutionary techniques, CMDPSOFS employs a crowding factor to maintain and update the leader set from generation to generation, which helps to filter out some crowded potential leaders. This mechanism will be more helpful for the datasets with a large number of features, where most non-dominated solutions in the leader set may have similar numbers of features and slightly different classification error rates. These crowded non-dominated solutions have a chance to be selected as a leader, which will limit the exploration ability of the algorithm. Eliminating such solutions helps the algorithm to explore the solution space more effectively to search for better results. Meanwhile, when selecting a leader for a particle, CMDPSOFS employs a binary tournament to select two non-dominated solutions from the leader set and the less crowded one will be selected as the leader. This mechanism attempts to keep the diversity of the

swarm in future iterations and further avoids particles from converging to local optima. Moreover, CMDPSOFS employs different mutation operators for different groups of particles to maintain the diversity of the swarm and balance its global and local search abilities. By contrast, NSPSOFS is less effective in terms of avoiding stagnation in local optima. NSPSOFS employs different levels of Pareto fronts to store the already found non-dominated solutions. Therefore, all the non-dominated solutions will be kept in the swarm from generation to generation. Such non-dominated solutions may be duplicated and the swarm may lose diversity quickly, which will lead to the problem of premature convergence. Although NSGAII employs the same mechanism to store the non-dominated solutions, NSGAII also employs mutation and crossover operators to keep the diversity. Therefore, NSPSOFS usually could not achieve as good results as CMDPSOFS and NSGAII.

G. How to Chose a Single Solution

In multi-objective problems, a set of Pareto front (non-dominated) solutions are obtained, which are trade-offs between different objectives. However, selecting a single solution from these solutions is an important issue. In feature selection problems, the two main objectives are minimising the number of features and maximising the classification performance, and the decision is a trade-off between these two objectives. If the Pareto front was “smooth” in that adding each additional feature would reduce the classification error rate by a small, but significant margin, users could weight the trade-off criteria to determine their preferred solutions. However, the results produced show that this is not usually the case. Adding features beyond a known limit number does not increase the classification performance. For example, the Musk1 dataset in Figure 2, the subset with the lowest classification error rate in CMDPSOFS-B has 40 features. Adding more features does not further increase the classification performance because it does not increase the relevance, but increases the redundancy and the dimensionality. Meanwhile, removing features may not lead to a decrease in classification error rate as relevant features may be removed. In the Musk1 dataset, the solution that stands in the “elbow” of CMDPSOFS-B would be a good choice. Therefore, visually seeing these possible solutions in the Pareto front assists users in determining their preferred compromises. This is actually the main reason why solving feature selection problems as multi-objective tasks is important.

VI. CONCLUSIONS AND FUTURE WORK

This work conducted the first study on multi-objective PSO for feature selection. Specifically, we considered two PSO based multi-objective feature selection algorithms, NSPSOFS and CMDPSOFS. The two feature selection algorithms were examined and compared with two conventional methods (LFS and GSBS), a single objective algorithm (ErFS), a two-stage method (2SFS) and three well-known multi-objective algorithms (NSGAII, SPEA2 and PAES) on twelve benchmark datasets of varying difficulty. Experimental results show that both NSPSOFS and CMDPSOFS can achieve more and better

feature subsets than LFS, GSBS, ErFS and 2SFS. NSPSOFS achieved similar (or slightly worse in some cases) results to NSGAII, SPEA2 and PAES in most cases. CMDPSOFS outperformed all other methods mentioned above not only on the number of features, but also on the classification performance. In particular, for the datasets with a large number of features, CMDPSOFS achieved better classification performance using fewer features and shorter computational time than the other four multi-objective algorithms.

This study finds that as multi-objective algorithms, NSPSOFS and CMDPSOFS can search the solution space more effectively to obtain a set of non-dominated solutions instead of a single solution. Examining the Pareto front achieved by the multi-objective algorithms can assist users in choosing their preferred solutions to meet their own requirements. Meanwhile, this work discovers that the potential limitation of losing the diversity of the swarm quickly in NSPSOFS limits its performance for feature selection. More importantly, this study highlights the benefits of the strategies of maintaining the diversity of the swarm in CMDPSOFS. A crowding factor together with a binary tournament selection can effectively select and filter out some crowded non-dominated solutions in the leader set. Different mutation operators in different groups of particles can effectively keep the diversity of the swarm and balance its global and local search abilities. These strategies accounts for the superior performance of CMDPSOFS over NSPSOFS, NSGAII, SPEA2 and PAES, especially on the datasets with large numbers of features.

The Pareto front multi-objective algorithms, NSPSOFS and CMDPSOFS, can achieve a set of good feature subsets, but it is unknown whether the achieved Pareto fronts can be improved or not. In the future, we will further investigate the multi-objective PSO based feature selection approach to better exploring the Pareto front of non-dominated solutions in feature selection problems. We will also intend to investigate the use of binary multi-objective PSO for feature selection and compare its performance with that of continuous multi-objective PSO. Meanwhile, we will also investigate whether using a given learning algorithm in a wrapper feature selection approach can select a good or near-optimal feature subset for other learning algorithms in the future.

ACKNOWLEDGMENT

This work is supported in part by the Marsden Fund of New Zealand Government (VUW0806 and 12-VUW-134), administrated by the Royal Society of New Zealand, and the University Research Fund (200457/3230) at Victoria University of Wellington.

REFERENCES

- [1] I. A. Ghelyas and L. S. Smith, “Feature subset selection in large dimensionality domains,” *Pattern Recognition*, vol. 43, no. 1, pp. 5–13, 2010.
- [2] M. Dash and H. Liu, “Feature selection for classification,” *Intelligent Data Analysis*, vol. 1, no. 4, pp. 131–156, 1997.
- [3] A. Unler and A. Murat, “A discrete particle swarm optimization method for feature selection in binary classification problems,” *European Journal of Operational Research*, vol. 206, no. 3, pp. 528–539, 2010.

- [4] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *The Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.
- [5] A. Whitney, "A direct method of nonparametric measurement selection," *IEEE Transactions on Computers*, vol. C-20, no. 9, pp. 1100–1103, 1971.
- [6] T. Marill and D. Green, "On the effectiveness of receptors in recognition systems," *IEEE Transactions on Information Theory*, vol. 9, no. 1, pp. 11–17, 1963.
- [7] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *IEEE International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995.
- [8] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *IEEE International Conference on Evolutionary Computation (CEC'98)*, pp. 69–73, 1998.
- [9] Y. Liu, G. Wang, H. Chen, and H. Dong, "An improved particle swarm optimization for feature selection," *Journal of Bionic Engineering*, vol. 8, no. 2, pp. 191–200, 2011.
- [10] A. Mohammed, M. Zhang, and M. Johnston, "Particle swarm optimization based adaboost for face detection," in *IEEE Congress on Evolutionary Computation (CEC'09)*, pp. 2494–2501, 2009.
- [11] J. Kennedy and R. Eberhart, "A discrete binary version of the particle swarm algorithm," in *IEEE International Conference on Systems, Man, and Cybernetics, 1997. Computational Cybernetics and Simulation*, vol. 5, pp. 4104–4108, 1997.
- [12] L. Y. Chuang, H. W. Chang, C. J. Tu, and C. H. Yang, "Improved binary PSO for feature selection using gene expression data," *Computational Biology and Chemistry*, vol. 32, no. 29, pp. 29–38, 2008.
- [13] C. L. Huang and J. F. Dun, "A distributed PSO-SVM hybrid system with feature selection and parameter optimization," *Application on Soft Computing*, vol. 8, pp. 1381–1391, 2008.
- [14] A. P. Engelbrecht, *Computational intelligence: an introduction* (2. ed.). Wiley, 2007.
- [15] R. Kohavi and G. H. John, "Wrappers for feature subset selection," *Artificial Intelligence*, vol. 97, pp. 273–324, 1997.
- [16] S. C. Yusta, "Different metaheuristic strategies to solve the feature selection problem," *Pattern Recognition Letters*, vol. 30, pp. 525–534, 2009.
- [17] S. Stearns, "On selecting features for pattern classifier," in *Proceedings of the 3rd International Conference on Pattern Recognition*, (Coronado, CA), pp. 71–75, 1976.
- [18] P. Pudil, J. Novovicova, and J. V. Kittler, "Floating search methods in feature selection," *Pattern Recognition Letters*, vol. 15, no. 11, pp. 1119–1125, 1994.
- [19] K. Kira and L. A. Rendell, "A practical approach to feature selection," *Assorted Conferences and Workshops*, pp. 249–256, 1992.
- [20] C. Cardie, "Using decision trees to improve case-based learning," in *Proceedings of the Tenth International Conference on Machine Learning (ICML)*, pp. 25–32, 1993.
- [21] H. Almuallim and T. G. Dietterich, "Learning boolean concepts in the presence of many irrelevant features," *Artificial Intelligence*, vol. 69, pp. 279–305, 1994.
- [22] B. Chakraborty, "Genetic algorithm with fuzzy fitness function for feature selection," in *IEEE International Symposium on Industrial Electronics (ISIE'02)*, vol. 1, pp. 315–319, 2002.
- [23] B. Chakraborty, "Feature subset selection by particle swarm optimization with fuzzy fitness function," in *3rd International Conference on Intelligent System and Knowledge Engineering (ISKE'08)*, vol. 1, pp. 1038–1042, 2008.
- [24] T. M. Hamdani, J.-M. Won, A. M. Alimi, and F. Karray, "Multi-objective feature selection with NSGA II," in *8th International Conference on Adaptive and Natural Computing Algorithms (ICANNGA'07) Part I*, vol. 4431, pp. 240–247, 2007.
- [25] M. Banerjee, S. Mitra, and H. Banka, "Evolutionary rough feature selection in gene expression data," *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 37, no. 4, pp. 622–632, 2007.
- [26] Z. X. Zhu, Y. S. Ong, and M. Dash, "Wrapper-filter feature selection algorithm using a memetic framework," *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, vol. 37, no. 1, pp. 70–76, 2007.
- [27] D. Muni, N. Pal, and J. Das, "Genetic programming for simultaneous feature selection and classifier design," *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, vol. 36, no. 1, pp. 106–117, 2006.
- [28] K. Neshatian and M. Zhang, "Genetic programming for feature subset ranking in binary classification problems," in *European Conference on Genetic Programming*, pp. 121–132, 2009.
- [29] H. H. Gao, H. H. Yang, and X. Y. Wang, "Ant colony optimization based network intrusion feature selection and detection," in *International Conference on Machine Learning and Cybernetics*, vol. 6, pp. 3871–3875, 2005.
- [30] H. Ming, "A rough set based hybrid method to feature selection," in *International Symposium on Knowledge Acquisition and Modeling (KAM '08)*, pp. 585–588, 2008.
- [31] X. Wang, J. Yang, X. Teng, W. Xia, and R. Jensen, "Feature selection based on rough sets and particle swarm optimization," *Pattern Recognition Letters*, vol. 28, no. 4, pp. 459–471, 2007.
- [32] G. Azevedo, G. Cavalcanti, and E. Filho, "An approach to feature selection for keystroke dynamics systems based on PSO and feature weighting," in *IEEE Congress on Evolutionary Computation (CEC'07)*, pp. 3577–3584, 2007.
- [33] S. W. Lin, K. C. Ying, S. C. Chen, and Z. J. Lee, "Particle swarm optimization for parameter determination and feature selection of support vector machines," *Expert Systems with Applications*, vol. 35, no. 4, pp. 1817–1824, 2008.
- [34] C. S. Yang, L. Y. Chuang, and J. C. Li, "Chaotic maps in binary particle swarm optimization for feature selection," in *IEEE Conference on Soft Computing in Industrial Applications (SMCIA '08)*, pp. 107–112, 2008.
- [35] L. Y. Chuang, S. W. Tsai, and C. H. Yang, "Improved binary particle swarm optimization using catfish effect for feature selection," *Expert Systems with Applications*, vol. 38, pp. 12699–12707, 2011.
- [36] H. N. A. Hamed, N. K. Kasabov, and S. M. Shamsuddin, "Quantum-inspired particle swarm optimization for feature selection and parameter optimization in evolving spiking neural networks for classification tasks," vol. 1, pp. 132–148, 2011.
- [37] M. A. Essegir, G. Goncalves, and Y. Slimani, "Adaptive particle swarm optimizer for feature selection," in *international conference on Intelligent data engineering and automated learning (IDEAL'10)*, (Berlin, Heidelberg), pp. 226–233, Springer Verlag, 2010.
- [38] B. Xue, M. Zhang, and W. N. Browne, "New fitness functions in binary particle swarm optimisation for feature selection," in *IEEE Congress on Evolutionary Computation (CEC'2012)*, pp. 2145–2152, 2012.
- [39] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [40] X. Li, "A non-dominated sorting particle swarm optimizer for multiobjective optimization," in *Annual conference on Genetic and evolutionary computation (GECCO'03)*, pp. 37–48, 2003.
- [41] M. R. Sierra and C. A. C. Coello, "Improving PSO-based multi-objective optimization using crowding, mutation and epsilon-dominance," in *EMO*, pp. 505–519, 2005.
- [42] M. Gutlein, E. Frank, M. Hall, and A. Karwath, "Large-scale attribute selection using wrappers," in *IEEE Symposium on Computational Intelligence and Data Mining (CIDM '09)*, pp. 332–339, 2009.
- [43] R. Caruana and D. Freitag, "Greedy attribute selection," in *International Conference on Machine Learning (ICML'94)*, pp. 28–36, 1994.
- [44] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength pareto evolutionary algorithm," in *Evolutionary Methods for Design, Optimization and Control with Applications to Industrial Problems*, pp. 95–100, 2002.
- [45] J. Knowles and D. Corne, "The Pareto archived evolution strategy: A new baseline algorithm for pareto multiobjective optimisation," in *Proceedings of the Congress on Evolutionary Computation (CEC'99)*, vol. 1, pp. 98–105, 1999.
- [46] A. Frank and A. Asuncion, "UCI machine learning repository," 2010.
- [47] T. Abeel, Y. V. de Peer, and Y. Saey, "Java-ml: A machine learning library," *Journal of Machine Learning Research*, vol. 10, pp. 931–934, 2009.
- [48] T. Hastie, R. Tibshirani, J. Friedman, and J. Franklin, "The elements of statistical learning: data mining, inference and prediction," *The Mathematical Intelligencer*, vol. 27, pp. 83–85, 2005.
- [49] F. Van Den Bergh, *An analysis of particle swarm optimizers*. PhD thesis, University of Pretoria, Faculty of Natural and Agricultural Science, Pretoria, South Africa, 2001.
- [50] A. Auger, J. Bader, D. Brockhoff, and E. Zitzler, "Theory of the hypervolume indicator: optimal distributions and the choice of the reference point," in *Proceedings of the tenth ACM SIGEVO workshop on Foundations of genetic algorithms*, FOGA '09, (New York, NY, USA), pp. 87–102, ACM, 2009.



Bing Xue received the BS degree in management in 2007 from Henan University of Economics and Law, Zhengzhou, China, and the MS degree in management in 2010 from Shenzhen University, Shenzhen, China. Since 2010, she has been working toward the Ph.D degree in computer science in Victoria University of Wellington, Wellington, New Zealand.

Her current research interests include evolutionary computation, particularly particle swarm optimisation, multi-objective optimisation, feature selection and dimension reduction for classification with high

dimensions.



Mengjie Zhang (SM'10) received a BE and an ME in 1989 and 1992 from Artificial Intelligence Research Center, Agricultural University of Hebei, China, and a PhD in computer science from RMIT University, Australia in 2000.

Since 2000, he has been working at Victoria University of Wellington, New Zealand. He is currently Professor of Computer Science and heads the Evolutionary Computation Research Group. His research is mainly focused on evolutionary computation, particularly genetic programming and particle

swarm optimisation with application areas of image analysis, multi-objective optimisation, job-shop scheduling, classification with unbalanced data, bioinformatics, and feature selection and dimension reduction for classification with high dimensions. He has published over 200 academic papers in refereed international journals and conferences. He has been serving as an associated editor or editorial board member for five international journals including IEEE TEVC, ECJ (MIT Press) and GPEM (Springer) and as a reviewer of over fifteen international journals. He has been serving as a steering committee member and a program committee member for over eighty international conferences. He has supervised over thirty postgraduate research students.

Prof Zhang is a senior member of IEEE, a member of the IEEE Computer Society, the IEEE CI Society and the IEEE SMC Society. He is also a member of the IEEE CIS Evolutionary Computation Technical Committee, a vice-chair of the IEEE CIS Task Force on Evolutionary Computer Vision and Image Processing, and a committee member of the IEEE New Zealand Central Section. He is a member of ACM and the ACM SIGEVO group.